CSE 2050: Algorithms and Data Structures II
Homework 05: Greedy Algorithms and Dynamic Programming
Total Points: 50

1 Ballot Boxes

Consider an election where there are n cities, with population $c_1, c_2, \ldots, c_n$, in a country where democracy is taken very seriously, and the entire population goes out to vote. The election commission assigns exactly k ballot boxes, $k \geq n$ for the election. It is clear that each city gets at least one ballot box. If multiple ballot boxes are assigned to a city, then the population of that evenly votes in these ballot boxes. For example, if the population is 100, and there are 3 ballot boxes, then 33, 33, and 34 people vote in these boxes. So, the largest number of people per ballot box is 34. To make sure that people don't wait in line for a long time to fulfill their democratic duty, we would like to distribute all k ballot boxes among the cities such that the largest number of people that vote in a single ballot box is minimized.

For example, if there are 4 cities with population $[120, 2680, 3400, 200]$, and there are 6 ballot boxes, then we would assign them as follows; $[1, 2, 2, 1]$, and the largest number of people voting in a ballot box is 1700. Any other assignment of boxes would not minimize this quantity.

Please write a simple program `ballot-boxes.py` that solves this problem. The constraints on the input are $1 \leq n \leq 100,000$, and $n \leq k \leq 500,000$. So, you need to find an efficient solution to this problem.

1.1 Input and Output

The input begins with two numbers on the first line, which are n and k. Then n lines follow, where each line is the population of a city. A population of a given city does not exceed $1,000,000$. Please output a single integer that gives the maximum amount of people voting in a single ballot box in an optimal assignment of ballot boxes. See sample input and output below.

```
Sample Input:

4 6
120
2680
3400
200

Sample Output:

1700
```

2 Edit Distance

The *edit distance* between two strings is the minimum number of operations required to transform one string into the other.

The allowed operations are:

- Add on character to the string.
- Remove one character from the string.
- Replace one character in the string.

For example, the edit distance between "LOVE" and "MOVIE" is 2, because you can first replace 'L' with 'M', and then add 'I'.

Given two strings as input, your task is to compute the edit distance between the two strings, and also to print a transcript of each transformation.

2.1 Input and Output

The first line of the input is a string containing m characters between `A-Z`. This is the start string. The second line of the input is a string containing n characters between `A-Z`. This is the end string. Both m and n are at most 1000.

As output, please print a single integer x that is the minimum number of operations needed to transform the input word to the output word. Then x lines follow each line containing two strings `p` and `q` in the format `p --> q`. The string `p` is the string that transforms into `q` in one operation. String `q` should contain the operation applied to a position character in `p`. The operations are indicated as follows:

- Indicate with '`[c]`' if the character at some position in added by the character `c`.
- Indicate with '`[*]`' if the character at some position is deleted.
- Indicate with '`.c.`' if the character at some position is replaced.

Note that the first string `p` on the first line is the input string `a`. After x lines of transormations are printed, print one line containing the output string `b`. See below for sample input and output.

```
Sample Input:

LOVE
MOVIE

Sample Output:

2
LOVE --> LOV[I]E
```

```
LOVIE --> .M.OVIE
MOVIE

Sample Input:

CATS
CAT

Sample Output:

1
CATS --> CAT[*]
CAT
```

3 Shopping Offers

In a certain shop, each kind of product has an integer price. For example, the price of a flower is 2 digital currencies (dc) and the price of a vase is 5dc. In order to attract more customers, the shop introduces some special offers, which consist of one or more product items together for a reduced price, also an integer. So, the shop may offer three flowers for 5dc instead of 6dc, or two vases together with one flower for 10dc instead of 12dc.

Write a program that calculates the price a customer has to pay for a purchase, making optimal use of the special offers to make the price as low as possible. You are not allowed to add items, even if that would lower the price. For the prices and offers given above, the (lowest) price for three flowers and two vases is 14dc.

3.1 Input and Output

The input starts with a singe integer s, that is the number of special offers. We have at most 99 special offers in the input. The following s lines describes a special offer using several integers. The first integer n, $(1 \leq n \leq 5)$, gives the number of products that are offered. The subsequent n pairs of integers c and k indicate that k items, $(1 \leq k \leq 5)$, with product code c, $(1 \leq c \leq 999)$, are part of the offer. The last number p on the line stands for the reduced price $(1 \leq p \leq 9999)$. The reduced price of an offer is less than the sum of the regular prices. The following line contains the number b, $(0 \leq b \leq 5)$, of different kinds of products to be purchased. Each of the following b lines contains three values: c, k, and p. The value c is the (unique) product code $(1 \leq c \leq 999)$. The value k indicates how many items of this product are to be purchased $(1 \leq k \leq 5)$. The value p is the regular price per item $(1 \leq p \leq 999)$. At most 25 items can be in the basket.

Output a single integer, that is the lowest possible price to be paid for the purchases. See sample input and output below.

```
Sample Input:

2
1 7 3 5
```

```
2 7 1 8 2 10
2
7 3 2
8 2 5

Sample Output:

14
```

4 Airline Prices

Plane ticket prices fluctuate wildly from one week to the next, and their unpredictability is a major source of frustration for travelers. Some travelers regret buying tickets too early when the prices drop right after they purchase the tickets, and some travelers regret buying tickets too late when prices rise right before they are about to make the purchase. At the end, no one is happy, except the airlines, of course.

Surely there is some reason to this madness. It turns out that airlines price their tickets dynamically, based on how many seats are still available and how close the flight is. For example, if there are very few seats left on a flight then the tickets may be expensive until the last few weeks before the flight, at which point the prices may decrease to fill the empty seats. Ultimately, the airlines wish to maximize revenue from each flight.

The airlines have collected and analyzed historical data, and have good estimates on the number of seats that will be sold at a particular ticket price with a particular number of weeks before the flight. Given the number of seats left on a flight as well as the number of weeks left before the flight, your job is to set the ticket price for the current week, in order to maximize the total revenue obtained from ticket sales from the current week to the time of the flight. You may assume that the number of tickets sold is exactly the same as the estimates, unless there are not enough remaining seats. In that case, all remaining seats will be sold. You may also assume that the optimal ticket prices will be chosen for the remaining weeks before the flight.

Note that higher prices do not necessarily mean fewer tickets will be sold. In fact, higher prices can sometimes increase sales as travelers may be worried that the prices will rise even higher later.

4.1 Input and Output

The first line of the input contains two integers n, and w, the number of seats left and the number of weeks left before the flight ($0 < m \leq 300, 0 \leq w \leq 52$). The next $w + 1$ lines give the estimates for 2 weeks, $w - 1$ weeks, ..., and down to 0 weeks (i.e. last week) before the flight. Each of these lines starts with an integer k, ($0 < k \leq 100$), the number of different prices to consider that week. This is followed by integers $0 < p_1 < p_2 < \ldots < p_k < 1000$ giving the prices in dollars. Finally, this is followed by k additional integers $s_1, \ldots, s_k$, ($0 \leq s_i \leq n$) indicating the number of tickets that will be sold for the corresponding prices.

On the first line, print the maximum total revenue the airline can obtain from ticket sales from the current week to the time of the flight. The next w lines should contain the optimal price for w weeks before the flight, $w - 1$ weeks before the flight, and so on up to the week of the flight (week

0). If there are multiple sets of ticket prices achieving this maximum, choose the smallest ticket price for the corresponding week.

```
Sample Input:

50 2
1 437 47
3 357 803 830 13 45 46
1 611 14

Sample Output:

23029
437
830
611

Sample Input:

100 3
4 195 223 439 852 92 63 15 1
2 811 893 76 27
1 638 3
1 940 38

Sample Output:

83202
852
811
638
940
```

5 Submission

Please submit all the individual files to `Canvas`. Please name your programs as given in the problem description, or refer to the names in the following table.

Problem Name	**File Name**
Ballot Boxes	`ballot_boxes.py`
Edit Distance	`edit_distance.py`
Shopping Offers	`shopping_offers.py`
Airline Prices	`airline_prices.py`